

QUY CHUẨN PHÁT TRIỂN & QUY TRÌNH KỸ THUẬT (DEVELOPMENT STANDARDS)

Hệ Sinh Thái SaaS Multi-Tenant Vahiztech

1. QUY CHUẨN LẬP TRÌNH JAVA (SPRING BOOT & QUARKUS)

1.1. Phiên bản & Tính năng Ngôn ngữ

- **Java 21 LTS** là chuẩn bắt buộc cho toàn bộ các dịch vụ Backend.
- Khuyến khích sử dụng tối đa các tính năng hiện đại:
 - **Java Records**: Dùng làm DTO (Data Transfer Objects), Value Objects và Event Payloads.
 - **Pattern Matching for Switch**: Xử lý phân loại sự kiện đa hình.
 - **Virtual Threads (Project Loom)**: Tận dụng trong Spring Boot
(`spring.threads.virtual.enabled=true`) để tối ưu xử lý I/O tác vụ nặng.
 - **Sealed Interfaces / Classes**: Định nghĩa danh mục trạng thái hữu hạn (Finite States).

1.2. Cấu trúc Phân tầng Mã nguồn (Clean / Hexagonal Architecture)

Mọi Microservice phải tuân thủ nghiêm ngặt mô hình 4 tầng:

```
com.vahiztech.[service]
├─ api/           # Controller REST Endpoints, SSE Resources, Request/Response DTOs
├─ domain/        # Core Business Logic, Entities, Domain Events, Exceptions (Độc lập Framework)
├─ application/   # Service Interfaces & Implementations, Use Cases, Transaction Boundaries
└─ infrastructure/ # Repository JPA/Panache, Keycloak Client, Kafka Producers/Consumers
```

1.3. Quy tắc Bảo mật Multi-Tenancy Bắt buộc

1. **Tuyệt đối không hard-code Tenant ID**: `tenant_id` phải luôn được trích xuất động từ `SecurityContextHolder` (Spring) hoặc `JsonWebToken` (Quarkus) qua `TenantContext`.
2. **Double Protection**: Mọi câu lệnh cập nhật hoặc xóa dữ liệu đều phải kiểm tra điều kiện `tenant_id`:

```
// ĐÚNG:  
taskRepository.findByIdAndTenantId(taskId, tenantId)  
    .orElseThrow(() -> new ResourceNotFoundException("Task không tồn tại trong tổ chức"))  
  
// SAI NGHIÊM TRỌNG (Nguy cơ rò rỉ chéo):  
taskRepository.findById(taskId);
```

3. **Phân quyền tại Tầng Phương thức:** Sử dụng `@PreAuthorize` kết hợp cả `Role` và `License` trên mọi Service / Controller method.

2. QUY CHUẨN FRONTEND (NEXT.JS & TYPESCRIPT)

2.1. Chuẩn TypeScript & ESLint

- Bật `strict: true` trong `tsconfig.json`. Tuyệt đối không dùng kiểu `any` mà phải khai báo Interface / Type rõ ràng.
- Đặt tên file theo chuẩn:
 - React Component: `PascalCase.tsx` (VD: `CourseCard.tsx`).
 - Hooks: `useCamelCase.ts` (VD: `useTenantSession.ts`).
 - Utilities & Services: `kebab-case.ts` (VD: `tenant-api-client.ts`).

2.2. Kiến trúc Next.js App Router

- Server Components mặc định:** Toàn bộ các trang (pages) và components phải là Server Components để giảm tải bundle JavaScript client và bảo vệ bí mật API.
- Client Components có chọn lọc:** Chỉ gắn `"use client"` khi component cần xử lý:
 - React Hooks (`useState`, `useEffect`, `useContext`).
 - Lắng nghe Event DOM (click, onChange, drag-and-drop Kanban).
 - Kết nối Real-time SSE / WebSockets.

3. CHUẨN HÓA PHẢN HỒI HTTP (UNIFIED API RESPONSE ENVELOPE)

Tất cả các REST API phải phản hồi theo cấu trúc thống nhất:

```
{
  "success": true,
  "data": { ... },
  "error": null,
  "timestamp": "2026-09-18T13:45:00Z",
  "traceId": "c85d8f74-3fa0-4e4b"
}
```

Khi xảy ra lỗi (`success: false`):

```
{
  "success": false,
  "data": null,
  "error": {
    "code": "RESOURCE_FORBIDDEN",
    "message": "Bạn không có quyền truy cập dữ liệu của Tenant khác",
    "details": ["Required tenant: tenant-alpha, requested: tenant-beta"]
  },
  "timestamp": "2026-09-18T13:45:00Z",
  "traceId": "c85d8f74-3fa0-4e4b"
}
```

4. QUY TRÌNH GIT FLOW & CONVENTIONAL COMMITS

4.1. Chiến lược Phân nhánh (Git Branching Model)

- `main`: Nhánh chạy trực tiếp trên môi trường Production. Mã nguồn chỉ được merge từ `release/*` hoặc `hotfix/*`.
- `develop`: Nhánh tích hợp chính cho toàn bộ tính năng của Sprint hiện hành.
- `feature/[sprint]-[tên-tính-năng]`: Nhánh phát triển của lập trình viên (VD: `feature/sp1-tenant-rls-schema`).
- `bugfix/[mã-lỗi]`: Nhánh sửa lỗi phát hiện trong quá trình kiểm thử.

4.2. Định dạng Commit Message (Conventional Commits)

<type>(<scope>): <mô tả ngắn gọn bằng thể mệnh lệnh>

[Nội dung chi tiết nếu có]

[Mã tham chiếu Task: VD VihoTask #IAM-12]

Các `type` được chấp nhận:

- `feat`: Thêm tính năng mới (Feature).

- **fix**: Sửa lỗi (Bug fix).
- **docs**: Thêm hoặc cập nhật tài liệu.
- **refactor**: Tái cấu trúc code nhưng không thay đổi hành vi nghiệp vụ.
- **test**: Thêm hoặc sửa đổi Unit/Integration tests.
- **chore**: Cập nhật cấu hình build tool, dependencies hoặc docker.

Ví dụ:

```
feat(security): implement CustomJwtAuthenticationConverter for tenant claims  
fix(database): enforce composite index on courses table for multi-tenant query  
docs(openapi): add VihoTask kanban board endpoints specification
```

4.3. Tiêu chí Kiểm duyệt Merge Request (PR Review Checklist)

1. Có ít nhất 1 Senior Developer phê duyệt (Approved).
2. Toàn bộ Unit Tests và Integration Tests trên CI chạy thành công 100%.
3. SonarQube Quality Gate không có lỗi Blocker hoặc Critical.
4. Xác nhận không có truy vấn cơ sở dữ liệu nào thiếu điều kiện **tenant_id**.